

HWS640 Homework #4 Writeup Created By Roz Roberts IV

Introduction

Overall this homework was quite difficult for me.

I have never been asked to make a CLI or LSTM model before, so this was quite a departure from my comfort zone. However, I can definitively say that I now know a lot more about how back and front end development actually work. This homework has single-handedly changed how I look at coding going forward as well as changing how I will be organizing my coding projects from here on. The rest of this document outlines my trials and successes through the last three weeks of development.

Methods Overview and Answers to Questions

Problem 1 Answers

LSTM or long short term memory recurrent neural networks are designed to learn from time-series like data like the previous work we did with the linear regression models. However, rather than "forgetting" long term data the LSTM also handles the long-term dependencies. The model that has been implemented in this homework works by using five input variables (prcp, tmax, tmin, srad, vp) and window to predict a 1-day out streamflow (qobs) value. The overall data set has data for 30 years from 1980 to 2010. The LSTM that has been implemented also ignores static attributes, this was done for simplicity. Over the 50 basins the data was split into three groups: training which comprised 70% of the basins, validation which comprised 15% of the basins,

and finally testing which comprised the final 15% of the basins. This ensured zero leakage of one group into another group. If the percentages didn't quite line up evenly the "remainder" basins were put with the testing group rather than the validation group. This split looks like 35 training basins, 7 validation basins, and 8 testing basins. I didn't split by time as I felt that would also complicate the LSTM. In addition to all of this some preprocessing was done, such as removal nan values, normalizing the datasets, and filling both forward and backward in time. This was to make sure that the data values were all similar and ready for direct model usage.

Problem 2 Answers

The overall architecture is as follows:

Input Sequence (batch, sequence length, 5) -> the 5 input variables are precipitation, temperature maximum, temperature minimum, solar radiation, vapor pressure

↓

LSTM Layer (hidden size default of 64, number of layers default 1) -> layers and hidden size can be specified

↓

Final timestep hidden representation (batch, 64)

↓

Fully connected linear layer -> takes the 64 points and converts it to a single values

↓

Predicted stream flow (batch, 1)

As we have already seen in previous homeworks and lectures stream flow is strongly dependent on

the weather conditions preceding the day that is being forecasted for, NOT just the conditions on the day of.

By accepting multiple input variables we can get a good image of the weather that was preceding the

forecast day, this is the perfect use case for an LSTM. As LSTMs learn from time-series like data

thus stream flow prediction is exactly what LSTMs are good at learning. As the dataset is rather simplistic we don't need to make a large LSTM with multiple hidden layers, hence why the defaults

were chosen with the values they have.

One expected strength is that LSTMs can learn lagged hydrologic responses, from the fact that they

retain "memory" of the data through the memory cell. However, one weakness is that my LSTM doesn't explicitly include static variables or basin physics. Thus, large differences in basin response will mean that validation and testing may not be as accurate as it could be. This would be due to the fact the LSTM would learn the response rates on some types of basins but wouldn't be good at generalizing basin Responses.

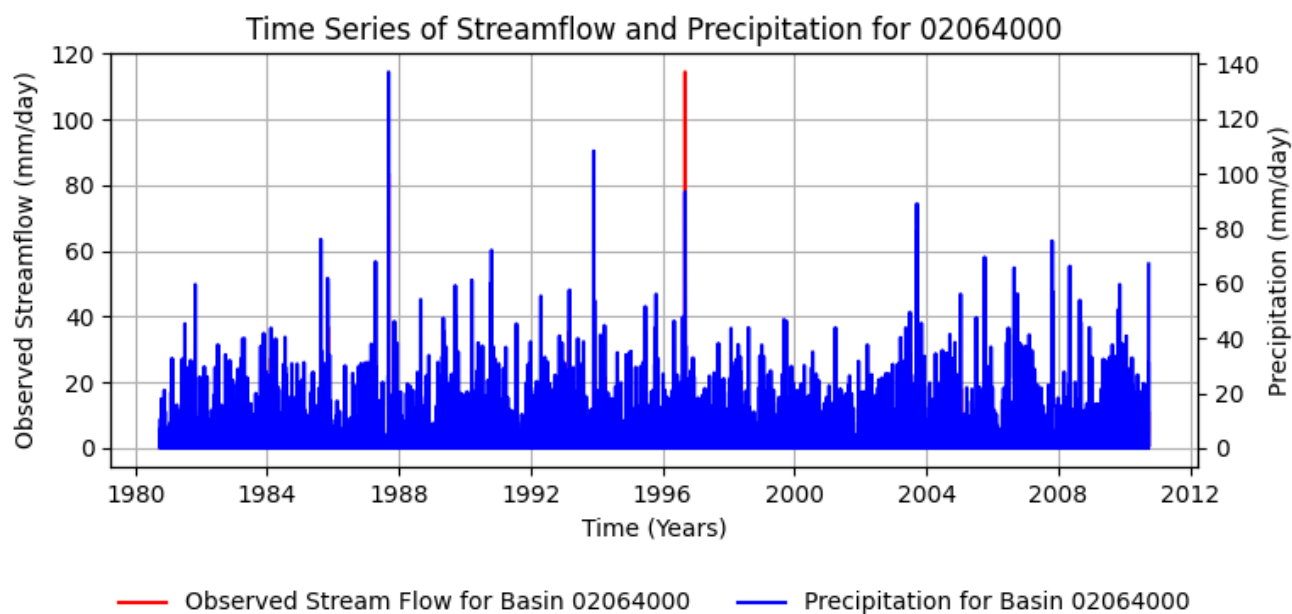
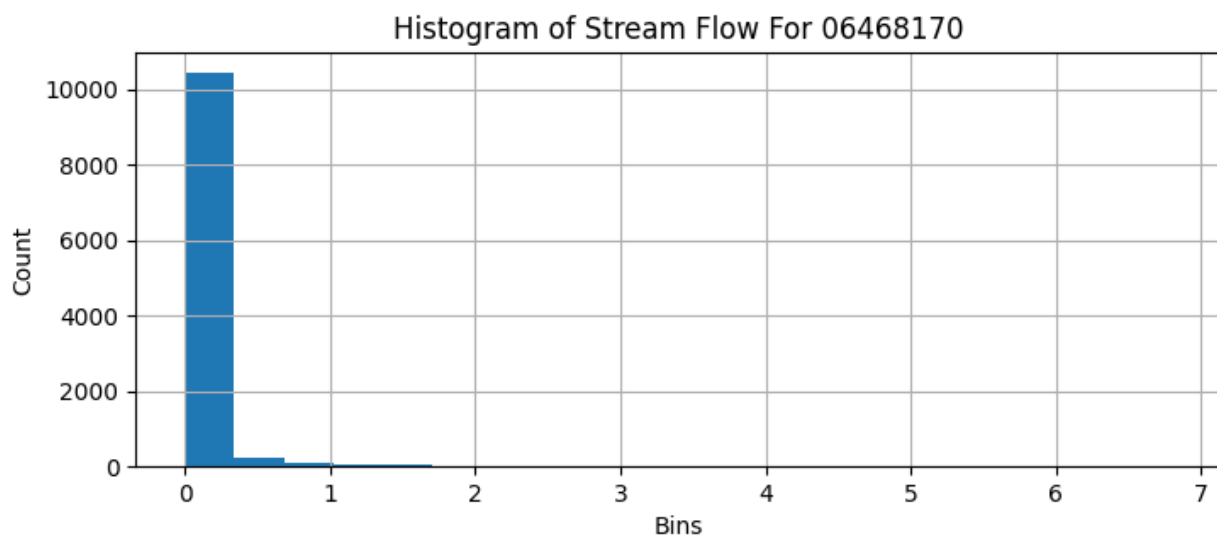
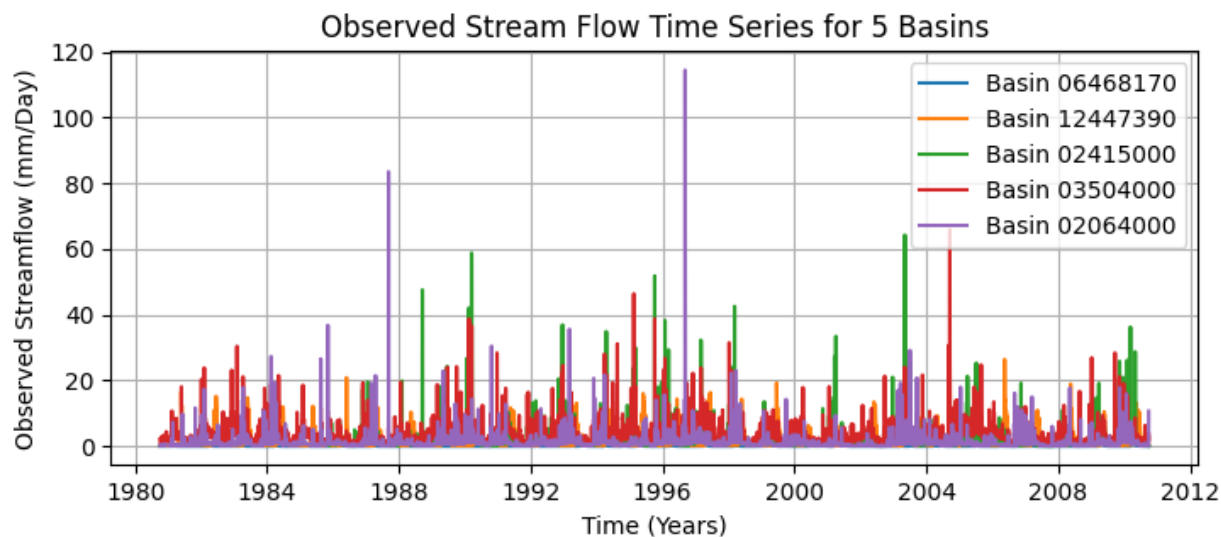
Problem 4 Answers

Basin 14316700 (STEAMBOAT CREEK NEAR GLIDE, OR) worked very well with the LSTM. Basin 0201600 (COWPASTURE RIVER NEAR CLIFTON FORGE, VA) didn't work as well as the above.

These two basins are very different in location, one being in Oregon and the other in Virginia. These locations are very different from each other in terms of geology. So my reasoning for the difference is the basin locations themselves as well as the basin dynamics. For example the first basin has many large influxes of stream flow throughout the dataset and the LSTM is able to capture that, however the second basin has many smaller influxes of streamflow that the LSTM has difficulty representing. These smaller (additive) influxes maybe where the LSTM architecture struggles as the large flow volume of Basin 1 is well represented. So my model may just be more suited to "flashy" type basins not more continuous.

Plots and Tables

Initial Exploration Plots and Structure Output



Total Dataset Structure:

Number of Basins: 50

Time Span of Dataset: 1980-10-01 to 2010-09-30

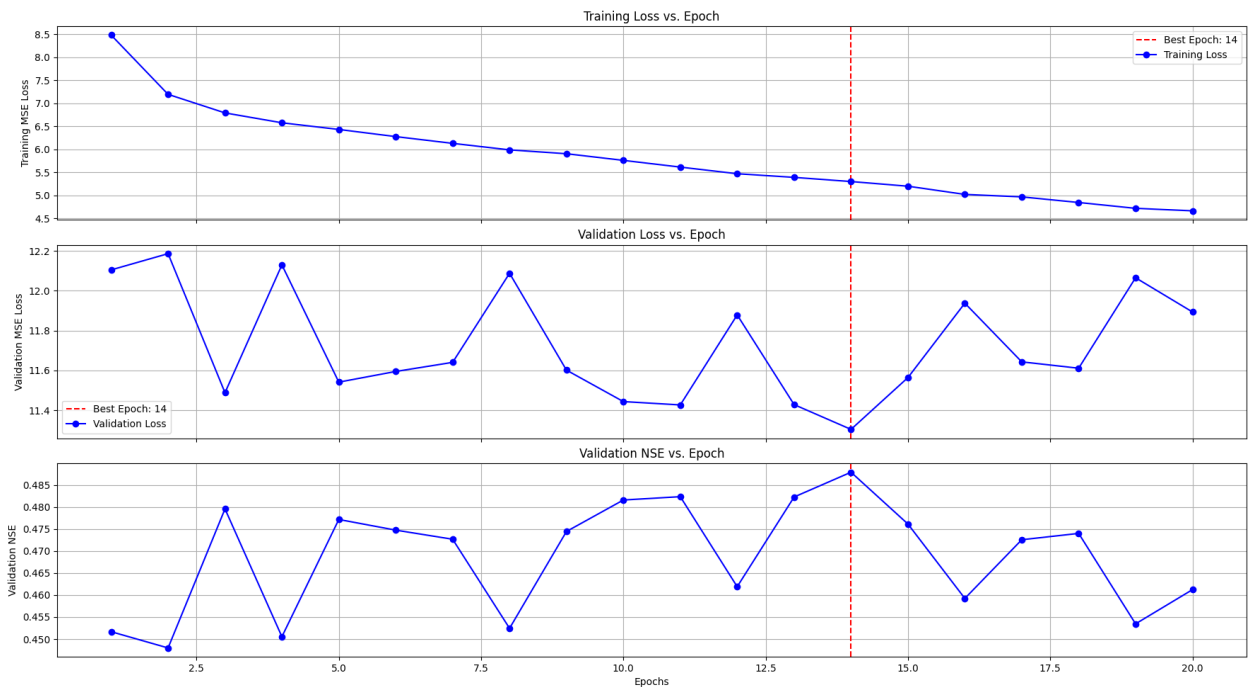
Dynamic Input Variables: prcp, tmax, tmin, srad, vp

Target Variable: qobs

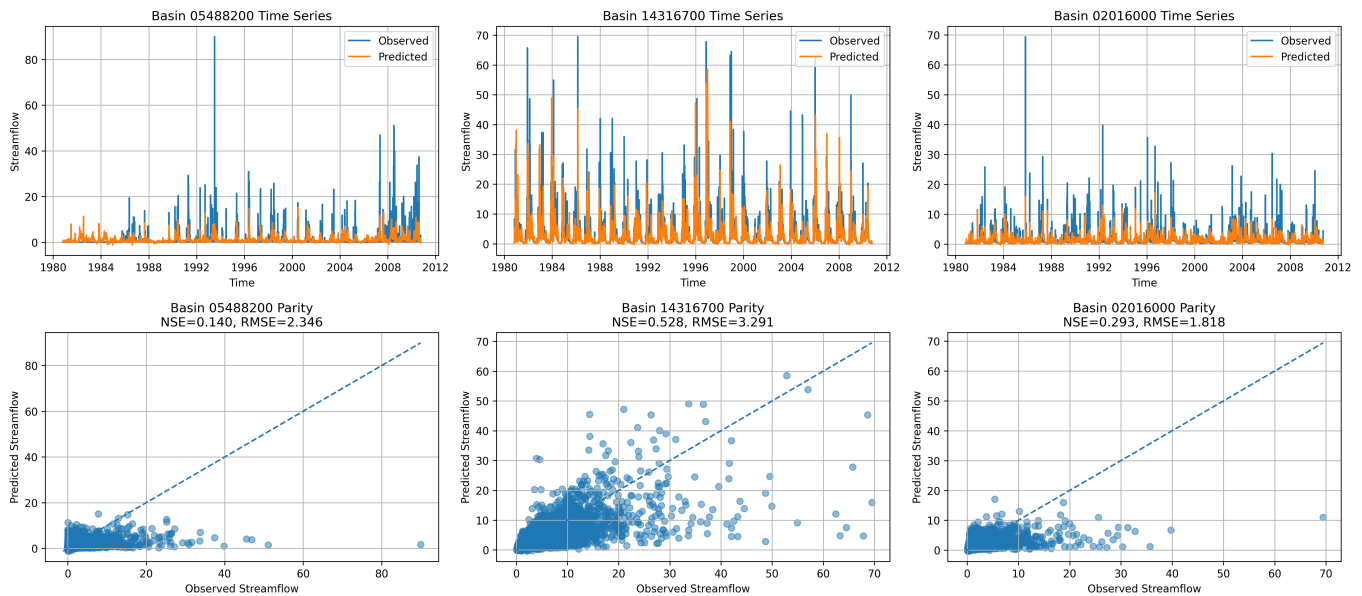
Number of Static Attributes: 16

Number of Static Attributes Used in LSTM: 0

Problem 3 Plot for best LSTM model



Problem 4 Plots and output qualitative metrics



Per-basin test metrics:

Basin 05488200

Basin Location: English Creek near Knoxville, IA

MSE: 5.503712

RMSE: 2.345999

MAE: 0.920816

NSE: 0.139721

Bias: 0.268200

Basin 14316700

Basin Location: STEAMBOAT CREEK NEAR GLIDE, OR

MSE: 10.832912

RMSE: 3.291339

MAE: 1.380612

NSE: 0.527581

Bias: -0.516944

Basin 02016000

Basin Location: COWPASTURE RIVER NEAR CLIFTON FORGE, VA

MSE: 3.304799

RMSE: 1.817911

MAE: 0.761515

NSE: 0.292555

Bias: 0.079878

Conclusion

Overall this homework was really enjoyable to undertake and I liked the challenges it presented. Although my model wasn't perfect, as it didn't generalize well, it did teach me many new things and I like that about this project. If I were to do this all over again from scratch, I would definitely include that static attributes in hopes of creating a better model result. In addition, I would

also play around with more combinations of different hyperparameters to see if that improved things.

Final Note: To reproduce my best results just run the repo with default parameters, as is explained in my README file.